

Day 6 — When the Answer Does Not Fit

Curriculum alignment: Stage 1 — Foundations of Computation · Signed Numbers and Bitwise Reasoning

Target time: ~2 hours (≈50 minutes reading, ≈70 minutes prediction and C experiments)

An eight-bit adder computes $255 + 1$. Mathematics says 256. The register says 0. Did the hardware make a mistake?

Why arithmetic has a boundary at all

Pure mathematics does not run out of integer digits. Physical machinery does.

A mechanical calculator has a finite number of wheels. A relay calculator has a finite number of relay groups. An electronic computer has finite registers and finite wires feeding its adder. Every additional bit costs components, space, propagation time, and energy.

The switching technology changed dramatically:

- mechanical contacts
 - electromechanical relays
 - vacuum tubes
 - transistors
 - integrated circuits containing many transistors

Each transition improved some combination of speed, size, reliability, and power. Relays physically moved and were slow. Vacuum tubes switched electronically but were large, hot, and failure-prone. Bell Labs researchers were looking for a smaller, lower-power solid-state replacement for telephone relays and tubes when Bardeen, Brattain, and Shockley produced the first working point-contact transistor in 1947.

The transistor eventually made millions and then billions of switching elements practical on one chip—but it did not make width free. A CPU designer must still choose register and datapath widths.

That is why overflow exists:

The mathematical answer is being forced back into a finite physical representation.

Flags are the CPU's way of preserving facts that the truncated result bits alone cannot tell us, such as whether an unsigned carry or signed overflow occurred.

1. Arithmetic happens inside a width

On paper, integers can grow as large as we like. Hardware registers have a fixed number of bits.

An eight-bit destination has only 256 patterns:

```
00000000 through 11111111
```

Now add:

```
  11111111
+ 00000001
-----
1 00000000
```

The full mathematical result needs nine bits. If the destination keeps only eight, it stores `00000000`; the ninth bit becomes a carry-out.

The adder did not calculate incorrectly. We asked it to produce a fixed-width result and then interpreted that result.

2. Unsigned arithmetic forms a clock

For an `n`-bit unsigned value, arithmetic behaves modulo `2n`.

Eight-bit unsigned:

```
255 + 1 ≡ 0 mod 256
0 - 1  ≡ 255 mod 256
```

Think of a clock with 256 positions:

```
... 253 → 254 → 255 → 0 → 1 → 2 ...
```

In C, arithmetic on unsigned integer types is defined modulo `2n` for the type's width. Wraparound is not undefined behavior.

This property is useful for:

- counters,
- hash functions,
- checksums,
- ring-buffer indices,
- cryptographic arithmetic,
- and sequence-number comparisons designed for wraparound.

Useful does not mean harmless. A wrapped allocation size can still become a security vulnerability if code assumes ordinary unbounded mathematics.

3. Signed overflow asks a different question

Eight-bit signed two's complement represents:

```
-128 through +127
```

Add:

```
01111111  +127
+ 00000001  +1
-----
10000000  interpreted as -128
```

The bit pattern is the correct low eight bits of binary addition, but `+128` is outside the signed range. Under the signed interpretation, the mathematical answer does not fit.

At the ISA level, CPUs usually report this with an **overflow flag**.

At the C-language level, signed integer overflow is **undefined behavior**. That distinction matters:

CPU behavior/flags -----	C abstract-machine rule -----
hardware produces low bits and updates status flags	compiler may assume signed overflow never occurs in a valid program

Do not reason “my CPU wraps, therefore signed C always wraps.” Optimizers use the language rule, not merely the observed behavior of one instruction.

4. Carry and overflow are not the same flag

The same adder supports two interpretations, so it reports two different conditions.

Carry flag: unsigned result did not fit

```
11111111 + 00000001 = 1 00000000
```

There is a carry out of the highest bit. Unsigned `255 + 1` does not fit.

Overflow flag: signed result did not fit

Signed overflow occurs when:

- two positive operands produce a negative-looking result, or
- two negative operands produce a positive-looking result.

Example:

```
01111111 (+127) + 00000001 (+1) = 10000000 (-128-looking)
```

No carry out is required for signed overflow.

Contrast:

```
11111111 + 00000001 = 1 00000000
  -1         +1         0
```

Here carry is set under the unsigned interpretation, but the signed result `-1 + 1 = 0` is perfectly valid. Overflow is clear.

Same bits, different validity test.

5. Detecting signed overflow from signs

For addition:

same-sign operands + different-sign result → signed overflow

The hardware can detect this from carry signals around the sign bit.

For subtraction:

different-sign operands + result sign differs from first operand
→ signed overflow

You do not need to memorize flag equations today. You need to understand why two flags exist: unsigned and signed encodings ask different range questions about the same low-bit result.

6. Sign extension: preserve value while widening

Suppose the eight-bit pattern is:

11110110 → -10

We want the same signed value in 16 bits.

Zero extension would produce:

00000000 11110110 → +246

That preserves the unsigned value, not the signed value.

Sign extension copies the old sign bit into every newly added high position:

11111111 11110110 → -10

Why does this work?

Eight-bit -10 is:

$$2^8 - 10 = 246$$

Sixteen-bit -10 is:

$$2^{16} - 10 = 65526$$

Copying ones into the new high bits adds exactly the place values required to move from the first modular representation to the second.

General rule:

unsigned widening → fill high bits with 0
signed widening → replicate the sign bit

7. Watch C perform both extensions

```
#include <stdint.h>
#include <stdio.h>

int main(void)
{
    uint8_t raw = 0xf6;
    uint32_t zero_extended = raw;
    int32_t sign_extended = (int8_t)raw;

    printf("raw:          0x%02x\n", raw);
    printf("zero-extended: 0x%08x = %u\n",
          zero_extended, zero_extended);
    printf("sign-extended: 0x%08x = %d\n",
          (uint32_t)sign_extended, sign_extended);
}
```

Predict the two 32-bit hexadecimal results before compiling.

Build:

```
gcc -std=c17 -Wall -Wextra -O0 -g extension.c -o extension
./extension
objdump -d -Mintel extension
```

Look for instructions such as `movzx` (move with zero extension) and `movsx` (move with sign extension). Exact code depends on optimization and surrounding operations, so verify rather than assume. As on Day 4, the conversion from out-of-range `uint8_t` to `int8_t` is implementation-defined under C17; this lab records GCC's behavior on the current two's-complement x86-64 target.

8. Run the overflow experiment safely

Open:

```
./daily/challenges/day-006-overflow-lab.c
```

It demonstrates:

- defined unsigned wraparound,
- checked signed addition,
- integer promotion surprises,
- sign extension,
- and compiler instrumentation for signed overflow.

Build normally:

```
gcc -std=c17 -Wall -Wextra -O0 -g \  
./daily/challenges/day-006-overflow-lab.c \  
-o /tmp/day-006-overflow  
  
/tmp/day-006-overflow
```

Then enable the undefined-behavior sanitizer:

```
gcc -std=c17 -Wall -Wextra -O1 -g \  
-fsanitize=undefined -fno-sanitize-recover=undefined \  
./daily/challenges/day-006-overflow-lab.c \  
-o /tmp/day-006-overflow-ubsan  
  
/tmp/day-006-overflow-ubsan
```

Predict where UBSan stops before running it.

9. A promotion trap

Consider:

```
uint8_t a = 255;  
uint8_t b = 1;  
printf("%d\n", a + b);
```

Does this print 0 or 256?

Usually 256. Before addition, both `uint8_t` operands undergo **integer promotion** to `int`, which can represent every `uint8_t` value on this machine. The addition therefore occurs at `int` width.

Assignment narrows afterward:

```
uint8_t result = a + b; /* 256 converted to uint8_t → 0 */
```

This is why “the variable is eight bits” does not automatically mean every intermediate expression is evaluated in eight bits.

Later C lessons will derive promotion rules carefully. Today, record the observation and its effect on experiments.

10. How production C checks overflow

Do not intentionally overflow a signed integer to test whether overflow happened; the test itself invokes undefined behavior.

Options include:

```
if (b > 0 && a > INT_MAX - b)  
/* positive overflow */
```

or compiler built-ins:

```
int result;
if (__builtin_add_overflow(a, b, &result))
    /* did not fit */
```

The built-in communicates intent and lets the compiler use hardware flags where available.

The Linux kernel also provides overflow-checking helpers. We will inspect their current implementations when the curriculum reaches kernel APIs.

11. Checkpoint

For each addition, use the same eight-bit result and answer two separate questions:

1. Did the unsigned result fit?
2. Did the signed result fit?

```
A. 11111111 + 00000001
B. 01111111 + 00000001
C. 10000000 + 11111111
D. 01000000 + 01000000
```

Then explain:

- why carry and overflow are different,
- why unsigned C wraparound is defined,
- why signed C overflow must not be assumed to wrap,
- and why sign extension copies the sign bit.

References used selectively

- Patt and Patel, *Introduction to Computing Systems*, 2nd ed., Chapter 2, arithmetic, sign extension, and overflow (book pages 27–33).
- Bryant and O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed., Chapter 2, §§2.2–2.3 on integer representations and arithmetic.
- ISO C semantics as reflected by GCC documentation for integer overflow, `-fsanitize=undefined`, and `__builtin_add_overflow`.
- Nokia Bell Labs, “The transistor: 75 years since the famed Nokia Bell Labs invention changed the world,” for the telephone-network motivation and the 1947 point-contact transistor: <https://www.bell-labs.com/institute/blog/the-transistor-75-years-since-the-famed-nokia-bell-labs-invention-changed-the-world/>

Next: Day 7 combines signed interpretation, masks, fields, overflow, and extension into one final topic-level mastery exercise.